MACHINE LEARNING PROJECT

Loan prediction dataset

(imported from Kaggle)

For this particular project, multiple dataset such as Logistic Regression, Decision Tree, Random Forest, and KNN were applied. The performance of the models was evaluated using accuracy, and the best-performing model was identified.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df=pd.read_csv('/content/train.csv')
print(df.head())
print(df.columns)
```

```
•••     Loan_ID Gender Married  ... Credit_History Property_Area Loan_Status
0  LP001002   Male      No  ...            1.0         Urban           Y
1  LP001003   Male     Yes  ...            1.0         Rural           N
2  LP001005   Male     Yes  ...            1.0         Urban           Y
3  LP001006   Male     Yes  ...            1.0         Urban           Y
4  LP001008   Male      No  ...            1.0         Urban           Y

[5 rows x 13 columns]
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

```python
print(df.isnull().sum())
print(df.info())
```

```
Loan_ID              0
Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount          22
Loan_Amount_Term    14
Credit_History      50
Property_Area        0
Loan_Status          0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            614 non-null    object
 1   Gender             601 non-null    object
 2   Married            611 non-null    object
 3   Dependents         599 non-null    object
 4   Education          614 non-null    object
 5   Self_Employed      582 non-null    object
 6   ApplicantIncome    614 non-null    int64
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         592 non-null    float64
 9   Loan_Amount_Term   600 non-null    float64
 10  Credit_History     564 non-null    float64
 11  Property_Area      614 non-null    object
 12  Loan_Status        614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None
```

```python
df['Gender'] = df['Gender'].fillna(df['Gender'].mode().iloc[0])
df['Married']= df['Married'].fillna(df['Married'].mode().iloc[0])
df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mode().iloc[0])
df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode().iloc[0])
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode().iloc[0])
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode().iloc[0])
df.isnull().sum()
```

	0
Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0

dtype: int64

```python
category_columns = ['Gender', 'Married', 'Dependents', 'Education',
                    'Self_Employed', 'Property_Area']

df = pd.get_dummies(df, columns=category_columns, drop_first=True)
print(df.head())
```

```
    Loan_ID  ApplicantIncome  ...  Property_Area_Semiurban  Property_Area_Urban
0  LP001002             5849  ...                    False                 True
1  LP001003             4583  ...                    False                False
2  LP001005             3000  ...                    False                 True
3  LP001006             2583  ...                    False                 True
4  LP001008             6000  ...                    False                 True

[5 rows x 16 columns]
```

```
y = df['Loan_Status']
x = df.drop(['Loan_Status', 'Loan_ID'], axis=1, errors='ignore')
```

```
x_train, x_test, y_train, y_test=train_test_split(x, y, test_size=0.2, random_state=42
)
```

```
scaler = MinMaxScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

```
lr = LogisticRegression(max_iter=1000)
lr.fit(x_train_scaled, y_train)
y_pred_lr = lr.predict(x_test_scaled)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
```

```
Logistic Regression Accuracy: 0.7886178861788617
```

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(x_train, y_train)
y_pred_dt = dt.predict(x_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.6991869918699187
```

```
rf = RandomForestClassifier(random_state=42)
rf.fit(x_train, y_train)
y_pred_rf = rf.predict(x_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.7804878048780488
```

```
knn = KNeighborsClassifier()
knn.fit(x_train_scaled, y_train)
y_pred_knn = knn.predict(x_test_scaled)

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

```
KNN Accuracy: 0.7398373983739838
```

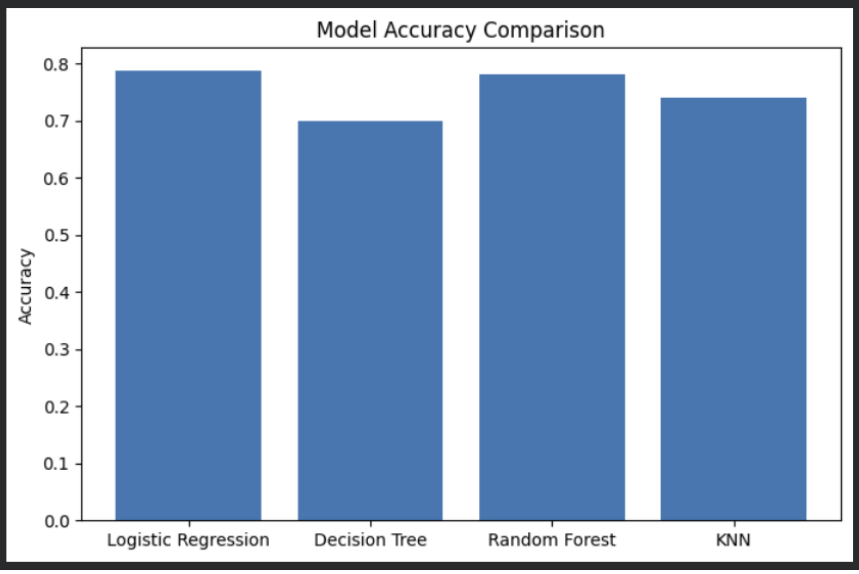
```python
print("LR Accuracy :", accuracy_score(y_test, y_pred_lr))
print("DT Accuracy :", accuracy_score(y_test, y_pred_dt))
print("RF Accuracy :", accuracy_score(y_test, y_pred_rf))
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

```
LR Accuracy : 0.7886178861788617
DT Accuracy : 0.6991869918699187
RF Accuracy : 0.7804878048780488
KNN Accuracy: 0.7398373983739838
```

```python
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
[[18 25]
 [ 2 78]]
              precision    recall  f1-score   support

           N       0.90      0.42      0.57        43
           Y       0.76      0.97      0.85        80

    accuracy                           0.78       123
   macro avg       0.83      0.70      0.71       123
weighted avg       0.81      0.78      0.75       123
```

```python
models = ['Logistic Regression', 'Decision Tree', 'Random Forest', 'KNN']
accuracies = [
    accuracy_score(y_test, y_pred_lr),
    accuracy_score(y_test, y_pred_dt),
    accuracy_score(y_test, y_pred_rf),
    accuracy_score(y_test, y_pred_knn)
]

plt.figure(figsize=(8,5))
plt.bar(models, accuracies)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```